To validate incoming API requests, it is recommended to use Laravel Form Requests. Create request classes using:

```
php artisan make:request StoreProductRequest
php artisan make:request UpdateProductRequest
```

In these classes, define the validation rules. Laravel automatically handles failed validation by returning JSON errors with the appropriate HTTP status code (422 Unprocessable Entity). Proper validation not only ensures data integrity but also improves the developer experience for API consumers.

14.7 Securing APIs with Laravel Sanctum

To protect your API, you can use Laravel Sanctum, which provides token-based authentication. First, install Sanctum:

```
composer require laravel/sanctum
```

Then publish the configuration and run the migrations:

```
php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"
php artisan migrate
```

Enable Sanctum by adding the middleware `auth:sanctum` to your API routes. Users can authenticate and receive a token, which must be included in the Authorization header for subsequent requests. Sanctum is ideal for single-page applications and mobile apps.

14.8 Pagination and Filtering

To handle large datasets, Laravel’s built-in pagination makes it easy to return results in chunks. You can paginate using:

```
return ProductResource::collection(Product::paginate(10));
```

You can also allow filtering by query parameters such as `name` or `price` range. This makes your API more flexible and user-friendly for front-end integration.

14.9 Best Practices in API Development

To build a robust API, it is important to follow industry best practices. Always return consistent JSON responses, use appropriate HTTP status codes, and validate all input data. Consider versioning your APIs using a prefix like `/api/v1` to maintain backward compatibility. Also, rate limiting can help prevent abuse by controlling the number of requests from a single client.